

TRAINING EFFICIENT LLMS

A six-month path from engineer to someone who trains LLMs — efficiently

Built on the book Efficient Training of LLMs — written from the experience of pre-training LatamGPT — plus interactive explorables, runnable notebooks, and pipelines you build yourself.

Omar U. Florez, PhD · omar-florez.github.io/training-efficient-llms

1 Foundations

2 Data

3 Training

4 Benchmarks

5 Ranking

1

Months 1–2

Foundations: how models are trained

WHAT YOU WILL DO

How learning works — loss, gradients, backpropagation — then language modeling and tokenization, and the transformer as a machine you can count. Every concept is implemented small before it is used big: each engineer writes the training loop, a BPE tokenizer, attention from scratch, and prices real models from their configs.

WHAT YOU WILL HAVE BUILT

A shared foundation across the team, with working implementations of the core mechanisms written by each engineer.

MATERIALS

BOOK — written

Ch. 1 Programs you don't write
Ch. 2 Language modeling
Ch. 3 The transformer, mechanically

INTERACTIVE

Gradient-descent playground · BPE stepper ·
Parameter counter · RoPE rotations

LABS

One Colab per chapter — run the book's numbers yourself

CH. 1

Programs you don't write

Loss, gradients, and the machine that turns data into functions

THE CHAPTER IN FIVE IDEAS

- You write the loss; the loss writes the program — the one mental shift the whole field rests on. Llama 2 7B is a function with 6.7B constants nobody wrote.
- The entire recipe in four lines — forward, loss, backward, update — run for real on a two-parameter rent model: loss 7,205,000 → 3,425 in ten steps, then the noise floor.
- Backpropagation is the chain rule over your call graph: the full gradient costs ~2 extra forward passes for any number of parameters — and its stored activations are what fill a training GPU.
- The optimizer minimizes training loss, which is a proxy: held-out data is the only honest measure, and at LLM scale the failure mode is benchmark contamination.
- Everything compiles to matrix multiplies — the hardware ladder from Python loops to an H100 spans ~7 orders of magnitude, and that is why GPUs exist.

6,738,415,616

constants in Llama 2 7B — not one chosen by a person. This chapter explains the machine that chose them.

GO DEEPER

Read: [chapters/ch01_programs.html](#)

Lab (Colab): [labs/ch01_programs.ipynb](#)

Play: [gradient-descent playground](#) — cause the divergence yourself

Language modeling

Tokens, cross-entropy, and the numbers you will stare at for months

THE CHAPTER IN FIVE IDEAS

- A tokenizer chops bytes into units by pure frequency (BPE): frequent strings become one token, rare ones stay in pieces — and an English-centric vocabulary taxes Spanish 30–50% more tokens per sentence.
- Embeddings turn ids into learned vectors whose geometry encodes similarity of usage — nobody defines similarity; prediction pressure does.
- The loss is one line: pay $-\log p$ of the true next token. Every position of every sequence trains at once; no labels are ever bought.
- Perplexity = e^{loss} is the effective branching factor: 50,257 at step zero, a cliff to ~50 on cheap statistics, then the expensive grind toward the entropy floor of language.
- Decoding (temperature, top-p) is not part of the model — the same weights are a careful assistant at $T=0.5$ and a brainstormer at $T=1.2$.

$$\ln 50,257 = 10.8$$

the first loss every GPT-2-vocabulary run prints, before learning anything — uniform guessing over the vocabulary.

GO DEEPER

Read: [chapters/ch02_language_modeling.html](#)
Lab (Colab): [labs/ch02_language_modeling.ipynb](#)
Play: BPE, one merge at a time

The transformer, mechanically

What each block does, what it costs, and how to count it by hand

THE CHAPTER IN FIVE IDEAS

- The residual stream is a bus: every component reads it, computes an update, and adds it back — attention moves information between positions, the MLP thinks per position.
- Attention is a soft dictionary lookup: queries dot keys, softmax, weighted sum of values — with a causal mask and a $\sqrt{d_k}$ that, if forgotten, silently ruins training.
- The MLP holds two-thirds of the parameters and works as a key-value memory: where the model's facts physically live.
- The modern block = the 2017 decoder minus cross-attention, plus four swaps: pre-norm, RMSNorm, RoPE, SwiGLU.
- Five config integers give the four facts: parameters $N(4d^2+3d\cdot d_{ff})+2Vd$ — 6.74B for Llama 2 7B, exact to the RMSNorm vectors — disk, FLOPs/token (6P), and KV cache (512 KB/token).

5 → 4

five integers in a config.json determine parameters, disk size, FLOPs per token, and KV-cache memory — computable in your head.

GO DEEPER

Read: [chapters/ch03_transformer.html](#)

Lab (Colab): [labs/ch03_transformer.ipynb](#)

Play: the parameter counter · RoPE rotations

2

A data generation & validation pipeline

Months 2-3

WHAT YOU WILL DO

Design and build a reproducible pipeline for producing training data — including synthetic instructions, dialogues, and reasoning traces — and for validating its quality: deduplication, quality filtering, factual checks, model-as-judge review, and human sampling. Dataset versioning and traceability from day one.

WHAT YOU WILL HAVE BUILT

An operating data pipeline with quality metrics and versioned datasets, ready to feed training.

MATERIALS

BOOK — in progress

Ch. 10 Data is the cheapest efficiency lever

Ch. 25 Synthetic data & distillation

FROM PHASE 1 YOU REUSE

The BPE stepper's compression lens: better data → fewer, denser tokens → compute saved

3

A training & fine-tuning pipeline

Months 3–4

WHAT YOU WILL DO

A standardized training pipeline on open models: supervised fine-tuning, LoRA/QLoRA, and DPO alignment as the use case requires — with experiment tracking, checkpoints, and GPU cost discipline. This is also where efficiency becomes architectural: what attention costs, why the KV cache rules serving, and how the labs shrank both.

WHAT YOU WILL HAVE BUILT

An end-to-end, reproducible fine-tuning pipeline, with at least one model adapted to a pilot domain.

MATERIALS

BOOK — written

Ch. 13 Making attention cheap
(FlashAttention · MQA · GQA · MLA · sliding windows)

INTERACTIVE

The Attention Zoo — five mechanisms, one decoding loop

IN PROGRESS

Ch. 16 Sequence packing · Ch. 22 SFT done cheaply

Making attention cheap

FlashAttention, MQA, GQA, MLA, sliding windows — and how to choose

THE CHAPTER IN FIVE IDEAS

- Attention presents three different bills: quadratic FLOPs (bind past $L^* \approx 6d \approx 25K$ tokens at 7B), the $L \times L$ training-memory artifact, and decode bandwidth — reading the whole KV cache per generated token at ~ 1 FLOP per byte.
- FlashAttention erases the memory artifact exactly — tiling through SRAM with an online softmax — and is simply always on.
- The cache shrinks by consolidating KV heads: MQA ($\div 32$, quality tax), GQA-8 (the industry default; reachable from MHA by a 5% uptrain), or DeepSeek's MLA — cache a 576-number latent, attend on it directly: 57 $\times$, quality intact.
- Sliding windows cap compute and cache but trade away direct long-range sight — so Mistral and Gemma interleave local and global layers.
- Decision: Flash always $\rightarrow$ GQA-8 by default $\rightarrow$ MLA when serving memory is the product $\rightarrow$ windows when context is long and local. Then evaluate long-range recall — the currency every technique quietly spends.

550 GB vs 13.5 GB

the KV cache versus the weights for an MHA 7B at 32K context, batch 32 — why this chapter exists.

GO DEEPER

Read: [chapters/ch13_attention.html](#)

Lab (Colab): [labs/ch13_attention.ipynb](#)

Play: the Attention Zoo — five mechanisms, one loop

4

A benchmarking pipeline

Months 4–5

WHAT YOU WILL DO

Build the benchmarks that measure what your users actually need — knowledge and reasoning in the domains that matter — and automate the evaluation: a benchmark harness, per-capability metrics, judge-based evaluation, and reports comparable across models and versions.

WHAT YOU WILL HAVE BUILT

A benchmark suite per domain and an automated evaluation pipeline any model can be run through.

MATERIALS

BOOK — planned

Ch. 9 How you'll know it's working
(benchmarks, contamination, cheap proxies,
building an eval suite you trust)

GROUNDWORK ALREADY LIVE

Ch. 2 §2.4 — loss, perplexity, and reading curves:
the health metrics everything else builds on

5

A living ranking of open models

Months 5–6

WHAT YOU WILL DO

Close the loop: a pipeline that periodically evaluates the leading open models — Llama, Qwen, Mistral, DeepSeek, Gemma, and whatever ships next — against your benchmark suite, weighing capability, inference cost, latency, license, and ease of deployment.

WHAT YOU WILL HAVE BUILT

A living model ranking with a dashboard and actionable model-selection guidance, maintained by the team that built it.

MATERIALS

BOOK — planned

Ch. 7 Scaling laws for training & inference

GROUNDWORK ALREADY LIVE

Ch. 13 Table 13.1 — attention at the frontier, model by model

The parameter counter — price any candidate model from its config in seconds

Everything lives in one place

omar-florez.github.io/training-efficient-llms

THE BOOK

Efficient Training of LLMs — chapters published as they are finished, in HTML and PDF. Written in second person, from the LatamGPT pre-training experience; every number computed for real.

INTERACTIVE

Six explorables: gradient descent, BPE, the parameter counter, RoPE, and the Attention Zoo — the same figures from the book, but you drive them.

LABS

One Colab notebook per chapter: run the code, reproduce the tables and plots, break things safely.

Materials marked *in progress* appear on the site as each phase approaches — the page is the single source of truth.